

Kisan Netra — Interview Prep Guide

The 30-Second Pitch (memorize this cold)

"I built Kisan Netra, a full-stack MERN application for farmers to track licensed pesticide and fertilizer inventory and find nearby dealers using geospatial search. It has JWT-based role authentication for farmers, dealers, and admins, OTP-based password reset, and a self-declared-plus-admin-verified trust model for dealer licensing, since there's no public government API for real-time verification in India. It's deployed on Render and Vercel with MongoDB Atlas."

Practice saying this without reading it. Interviewers often just say "walk me through your project" and stop talking — this is your opening.

EASY — Fundamentals they expect anyone to know

Q: What's your tech stack and why did you choose it? MongoDB, Express, React, Node — MERN. Chose Mongo because the data (dealers with geo-coordinates, nested usage logs) fits a document model well, and Mongo's native geospatial indexing (2dsphere) was a direct requirement, not something I'd bolt onto a relational DB.

Q: How does your authentication work? JWT signed on login/register, stored in an httpOnly cookie — not localStorage. httpOnly means client-side JS can't read it, which protects against XSS token theft. The cookie is sent automatically on every request via withCredentials: true in Axios.

Q: Why httpOnly cookies instead of localStorage for the token? localStorage is readable by any JavaScript running on the page — including injected scripts from an XSS vulnerability. An httpOnly cookie is invisible to JS entirely; only the browser can send it, and only the server can set/read it.

Q: What is RBAC and how did you implement it? Role-Based Access Control. I have three roles — farmer, dealer, admin — stored on the User model. An authorize(...roles) middleware checks req.user.role against an allowed list before letting a request through. E.g. only dealers can create a store profile, only admins can verify one.

Q: What's the difference between authentication and authorization? Authentication = confirming who you are (login, JWT verification). Authorization = confirming what you're allowed to do once we know who you are (RBAC middleware).

Q: Why bcrypt for passwords? It's a slow, salted hashing algorithm designed specifically to resist brute-force and rainbow-table attacks — unlike fast hashes like MD5/SHA-256 which are built for speed, the opposite of what you want for passwords.

Q: What's the purpose of your .env file? Keeps secrets (DB connection string, JWT secret, email credentials) out of source code. It's gitignored so it's never pushed to GitHub — I actually had to think about this directly when I moved my seed script's admin credentials out of hardcoded values into env vars.

MEDIUM — Design decisions, expect "why" follow-ups

Q: Walk me through your database schema. Core collections: User (auth + role), Dealer (store profile, linked 1:1 to a User), Product (master catalog), DealerStock (join table between Dealer and Product with quantity), FarmerInventory (a farmer's personal stock with an embedded usage log array).

Q: Why is DealerStock a separate collection instead of embedding stock inside Dealer? Because it's a genuine many-to-many relationship — one dealer carries many products, one product is carried by many dealers. Embedding an array of products inside each Dealer document would make "which dealers carry X" a full collection scan across every dealer's array. A separate join collection lets me query it directly and index it.

Q: Why did you embed the usage log inside FarmerInventory instead of a separate collection? Usage logs are always read and written in the context of one specific inventory item — I never need to query usage logs independent of their parent. That's the classic case for embedding over referencing: data that's always accessed together, with no need for independent queries.

Q: How does your geospatial search work? Dealer.location is a GeoJSON Point ([longitude, latitude] — note that's the required order, backwards from how people say it out loud). I put a 2dsphere index on it. The search endpoint uses a \$geoNear aggregation stage — not a plain find() with \$near — because \$geoNear returns an actual distance field per result, which a plain \$near query doesn't give you.

Q: How do you filter nearby search results by a specific product? Two-step: first I query DealerStock for which dealer IDs stock that product with quantity > 0, then I pass those IDs into the \$geoNear aggregation's query filter alongside isVerified: true. I don't try to do it all in one query because DealerStock and Dealer are separate collections without a \$lookup in the geo pipeline in my current version — a possible improvement would be adding a \$lookup stage instead of a separate pre-query.

Q: How does your OTP password reset work end to end? Three endpoints. forgot-password generates a 6-digit OTP, hashes it with bcrypt before storing (same as a password — never stored in plaintext), sets a 10-minute expiry, and emails it via Nodemailer. verify-otp checks the hash match, expiry, and a max-attempts counter (capped at 5, to prevent brute-forcing a 6-digit code). reset-password re-validates the OTP and updates the password hash.

Q: Why hash the OTP if it's only valid for 10 minutes anyway? Defense in depth — if the database leaks during that 10-minute window, I don't want raw OTPs sitting there next to user emails, ready to be used immediately.

Q: How do you prevent brute-forcing the OTP? An attempts counter on the user document, capped at 5 wrong guesses before the OTP is invalidated and a new one must be requested. Also rate-limited at the route level via express-rate-limit on the auth endpoints specifically, not just globally.

Q: Why did you rate-limit auth routes more strictly than the rest of the API? Login and forgot-password are the two endpoints an attacker actually wants to hammer — credential stuffing or OTP brute-forcing. A general limit across the whole API wouldn't stop someone from hitting login hundreds of times, so those specific routes get a tighter limit.

Q: How does your dealer verification / trust model work? Dealers self-declare a license number when creating their store profile. That store starts as `isVerified: false` and won't appear in farmer search results. An admin panel lists pending dealers; admins manually flip `isVerified` to `true` after reviewing. I chose this over trying to build a live government-API verification because I actually checked — no public real-time verification API exists for pesticide dealer licenses in India. It's a genuine constraint I designed around rather than faked.

Q: Why doesn't your nearby-search show unverified dealers? Trust and safety — an unverified store hasn't been checked by an admin, so surfacing it in search would undermine the whole point of a "licensed" platform. It's filtered directly in the `$geoNear` query's query clause.

Q: What's the difference between `sameSite` `strict`, `lax`, and `none`, and which did you use? `strict` only sends the cookie on same-site requests. `lax` allows it on top-level navigations cross-site but blocks it on cross-site XHR/fetch — which is exactly what broke my deployment. `none` allows cross-site sending but requires `secure: true` (HTTPS). Locally, frontend and backend share localhost, so `lax` works fine. In production, Vercel and Render are different domains, so I had to switch to `none` plus `secure: true`, conditional on `NODE_ENV`.

HARD — System design, tradeoffs, "what would you do differently"

Q: Your app currently computes alerts (low stock, expiring soon) on-request rather than via a background job. What's the tradeoff, and when would that stop scaling? On-request avoids infrastructure complexity — no cron/queue needed, and the data is always fresh since it's computed at read time. The tradeoff is it does a full query and filter every time the farmer opens their dashboard, which is fine at small scale but would get expensive with thousands of inventory items per farmer, or if I wanted to push notifications proactively rather than only showing alerts when the user happens to open the app. At that point I'd move to a scheduled job that computes alerts periodically and either caches them or triggers outbound notifications.

Q: How would you scale the nearby-dealer search if you had millions of dealers? The `2dsphere` index already handles the core geo lookup efficiently. The bigger risk is the pre-query step for product filtering. At large scale I'd convert that into an aggregation `$lookup` stage so it's done inside the database rather than round-tripped through the app, or consider a dedicated search service like Elasticsearch with geo-distance queries if filtering got much more complex than one product ID.

Q: What are the security risks in your current auth setup, and how would you harden it further? A few real ones: no email verification originally meant anyone could register with

a fake email, which is why I built an OTP verification flow. JWT has no revocation mechanism — if a token leaks, it's valid until expiry; a production system would want a token blocklist or short-lived access tokens with refresh tokens. The seed script's default admin password is a placeholder, which is why I moved it to an env var so it's never in git history. There's no CSRF protection beyond sameSite cookies — for a more sensitive app I'd add explicit CSRF tokens on state-changing requests.

Q: Explain a real bug you hit during deployment and how you debugged it. After deploying to Render (backend) and Vercel (frontend) — different domains — login succeeded but every subsequent request returned 401. I used the Network tab to see the pattern: login returned 200 and set a cookie, but the next request wasn't sending it back. That pointed to the cookie itself, not the auth logic. The cause was sameSite strict or lax, which doesn't get sent on cross-site XHR requests. Cross-domain deployments require sameSite none plus secure true, conditional on NODE_ENV production so local dev still works normally.

Q: You also hit a case-sensitivity bug — what happened? Locally on Windows, file systems are case-insensitive, so a require() path with mismatched casing still resolved fine. Render runs Linux, which is case-sensitive, so the exact same code that ran locally threw Cannot find module in deployment. It's a good example of why "works on my machine" isn't sufficient — the fix was matching the exact casing between the import path and the actual filename.

Q: If you were rebuilding this from scratch, what would you do differently? Three things: use \$lookup in the geo aggregation pipeline from day one instead of a separate pre-query for product filtering; introduce refresh tokens alongside the access token rather than a single long-lived JWT with no revocation path; write tests earlier in the build instead of consolidating them on a dedicated hardening day, since catching schema or API contract mismatches sooner would save later rework.

Q: How do you handle a race condition where two farmers try to log usage against the same dealer's stock simultaneously? Honestly, I don't currently have explicit concurrency control on that path. The correct fix is an atomic update using findOneAndUpdate with a \$inc operator, decrementing quantity server-side in a single atomic operation, plus a query condition like quantityRemaining greater than or equal to quantityUsed so the update itself fails safely if stock would go negative, rather than trusting an app-level check that could be stale by the time it writes.

Q: Why REST instead of GraphQL for this API? The scale and shape of the data didn't call for GraphQL's main advantages — my endpoints are fairly resource-shaped already, and I don't have deeply nested, client-varying query needs that make GraphQL worth its added complexity. REST with clear, well-scoped endpoints was the simpler correct choice here.

Q: How would you add real-time updates, so a farmer sees a dealer's stock update live? I'd add Socket.io, or Mongo Change Streams feeding a WebSocket layer, so a stock update emits an event to farmers currently viewing that dealer's page. Currently the app is fully

request and response — a farmer has to refresh to see updated stock, which is a reasonable v1 tradeoff but a clear next step.

Behavioral / Process Questions

Q: How did you approach planning this project? Locked down MVP scope, schema, and API endpoints before writing code — day one was planning only. Then built backend and frontend in parallel per feature, connecting and committing after each slice, rather than finishing the entire backend first, which is a common mistake that leads to redesigning APIs after the fact.

Q: How did you decide what to cut from scope? I explicitly wrote a "Future Work" section — multi-language support, SMS alerts, a payments flow, real government license verification — rather than half-building any of them. A scoped, working core beats a bloated, half-finished feature set.

Q: What was the hardest part of this project? Two candidates, be ready to pick one and go deep: the geospatial aggregation query, or the cross-domain cookie authentication bug during deployment. It's a strong answer because it shows real debugging process, not just reading docs.

Quick-Fire Terms to Have Crisp Definitions For

- 2dsphere index: MongoDB index type for GeoJSON data, enabling geospatial queries like \$near, \$geoWithin, \$geoNear.
 - Middleware: a function that runs between the request coming in and the route handler — used here for auth verification, RBAC, rate limiting, error handling.
 - Idempotency: an operation that produces the same result no matter how many times it's run — relevant for the seed script, which is safe to re-run because it checks for existing records first.
 - CORS: browser security mechanism restricting cross-origin requests; configured here via credentials true plus an exact-match origin, not a wildcard, since wildcards can't be combined with credentialed requests.
 - N+1 query problem: not directly hit in this project, but know the definition — looping and querying once per item instead of one batched query.
-

Questions to Ask Your Interviewer

- What does the team's approach to technical debt look like, how do you decide when to refactor versus ship?
 - What's the biggest scaling challenge your engineering team is working through right now?
 - How is code review typically structured on the team I'd be joining?
-

Final Reminders

- If you don't know something, say so honestly and reason through it out loud — interviewers care more about how you think than whether you have a memorized answer.
- Don't over-claim. If a feature is a stretch or future-work item, say that clearly rather than implying it's built.
- Have your GitHub repo and live demo link open and ready to share your screen if asked.